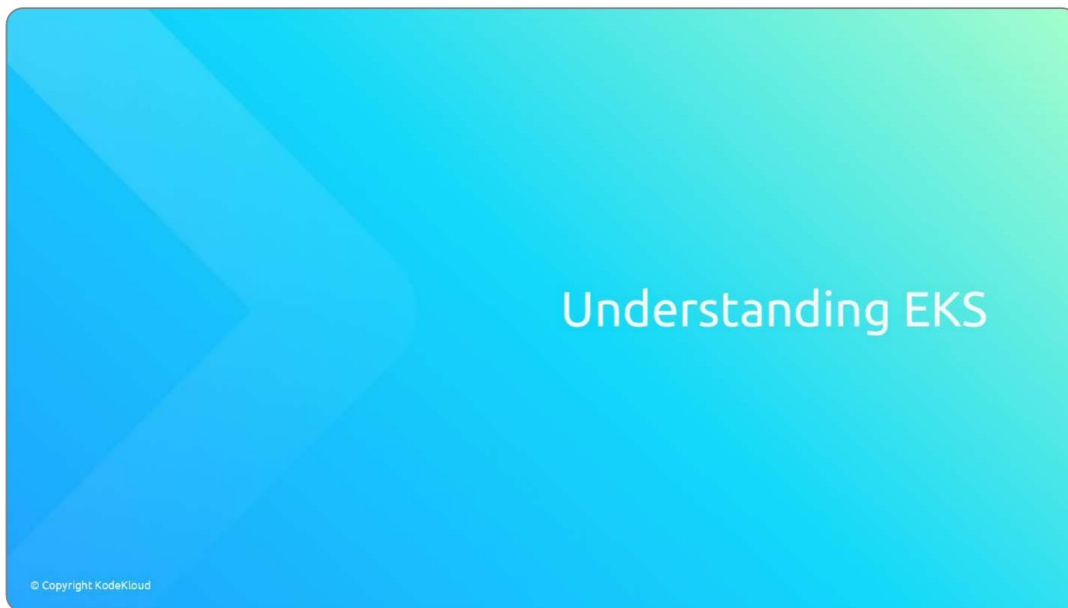




Understanding EKS

A student-friendly study guide to the full KodeKloud AWS EKS course

Rewritten as conversational notes, with original diagrams for every major concept.

What's inside

1. Understanding EKS — what it is & how it's built
2. Networking — the VPC CNI, IPs, and network policy
3. Storage — EBS, EFS, and secrets
4. Getting traffic in & out — Services, Load Balancers, Ingress
5. The Gateway API & VPC Lattice
6. Compute options — Fargate, Node Groups, Karpenter
7. Cluster access & authentication
8. IAM for pods — Kube2IAM, IRSA, Pod Identity
9. Security groups for pods
10. Observability & monitoring
11. Upgrading your cluster
12. EKS add-ons

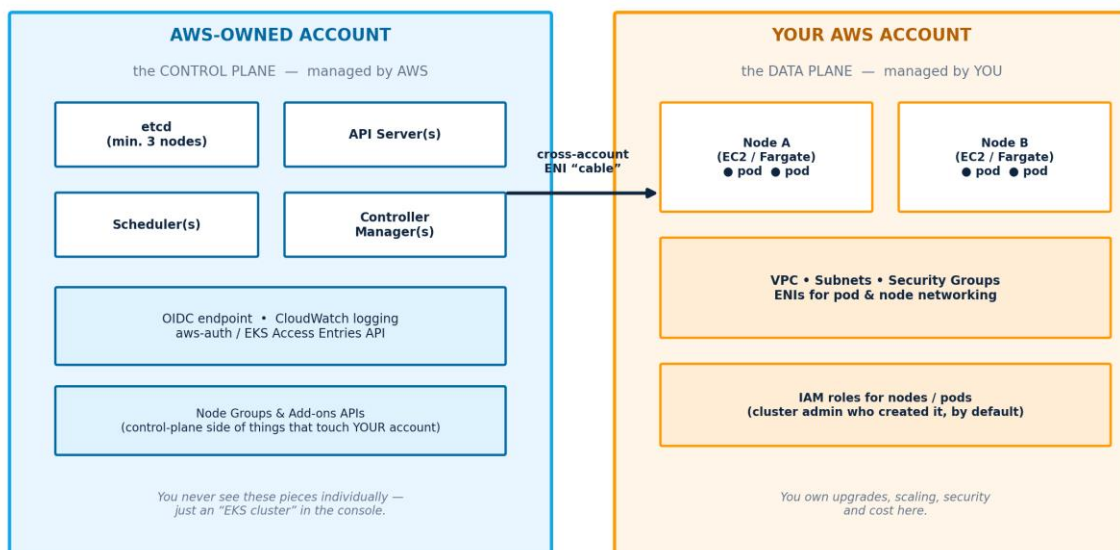
1. Understanding EKS

What it is, and how it's different from a normal Kubernetes cluster

Let's start with the basics: EKS is AWS's hosted service for Kubernetes. Every Kubernetes cluster, anywhere, is made of two halves — a control plane (etcd, the API server, the scheduler, the controller manager: the 'brain') and a data plane (the worker nodes that actually run your pods). With EKS, AWS draws a clean line right down the middle of that split:

- **Control plane** — lives in an AWS-owned account. AWS runs and patches it for you. You just see “an EKS cluster” in the console; you never see the individual etcd nodes or API servers.
- **Data plane** — lives in your own AWS account. You own the nodes, the VPC, the security groups, and (mostly) the upgrades.

The two sides are bridged by cross-account Elastic Network Interfaces (ENIs) — think of it as literally plugging a cable between two separate networks so they can talk.



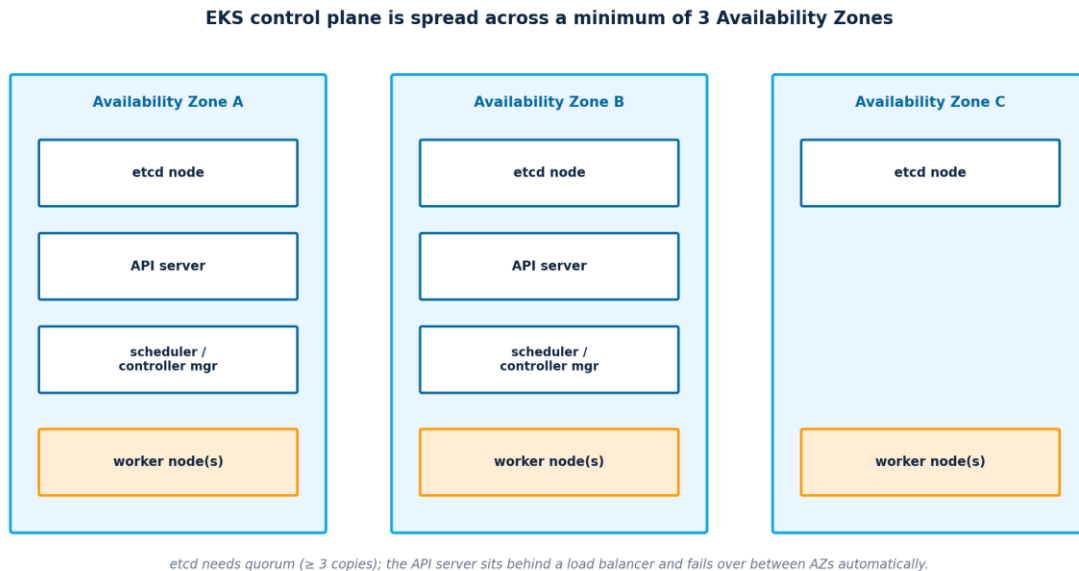
Why use EKS instead of running Kubernetes yourself on EC2?

You could absolutely run your own control plane on EC2 (tools like kops or kubespary have done this for years). You'd get full control — your own scheduler flags, your own upgrade schedule, any cluster size you want. The catch is you now own everything, including the scariest part: etcd. Etcd is the distributed database behind every Kubernetes cluster; if it's lost, your cluster is effectively gone. Many teams switch to a hosted service the first time they have to restore an etcd backup themselves. EKS hands that responsibility — and the 24/7 operational burden of keeping the control plane highly available — to AWS.

Most companies aren't running AWS just for Kubernetes — they're already using RDS, S3, load balancers, Route 53, etc. EKS is simply the easiest way to wire a Kubernetes cluster into the rest of that AWS ecosystem.

Inside the control plane

EKS is a regional service, but a region is really a group of Availability Zones (separate data centers). Your control plane pieces get spread across a minimum of three AZs so an outage in one doesn't take the cluster down — etcd needs quorum, so it always keeps at least three copies alive and running a leader election.



Beyond the standard Kubernetes pieces, an EKS control plane also quietly sets up a few EKS-specific extras:

- **OIDC endpoint** — authentication for anything that needs to call into AWS from inside the cluster (more in Section 8).
- **CloudWatch logging** — opt-in, but the plumbing is already there for you.
- **aws-auth / EKS Access Entries API** — maps AWS IAM identities to Kubernetes RBAC (more in Section 7).
- **Node Groups & Add-ons APIs** — technically part of the EKS control-plane API, but they actually create resources (nodes, workloads) inside your own account.

How do you actually create a cluster?

There's no single right answer — it comes down to what your team already knows:

- **Console** — works, but was consistently the most-complained-about experience because EKS touches so many other AWS services (IAM, VPC, load balancers) that you end up bouncing between tabs.
- **CloudFormation** (hand-written YAML, or generated via the CDK) — good if you're already an AWS-native shop.
- **Terraform** — probably the most popular choice. AWS even maintains a set of community modules called “EKS Blueprints.” You own your own state file and access, since it's not a managed service itself.
- **Everything else** — Pulumi, Cluster API, and a dozen other tools that ultimately just script the same CloudFormation/Terraform/CLI calls for you.

Whatever tool you pick, you'll need fairly broad IAM permissions to create a cluster — EKS touches EC2, VPCs, security groups, load balancers and IAM roles all at once. And don't forget the aws-iam-authenticator plugin for kubectl — the EKS API server sits behind a load balancer that requires AWS-signed credentials, so this plugin is what turns your IAM identity into something kubectl can use to log in.

2. Networking

The VPC CNI, IP addressing, and network policy

Every node and every pod in an EKS cluster pulls its IP address straight from your VPC's subnets — and because subnets are tied to a single Availability Zone, so is every node and pod that lives in them. Nodes (and pods) reach the network through Elastic Network Interfaces (ENIs), which behave just like a physical network card being plugged into a server.

The ENI / IP-address bottleneck

Here's the catch: EC2 instance types have hard limits on how many ENIs they can attach, and how many IP addresses each ENI can hold. A small instance might only support 4-5 ENIs at 4-5 IPs each. Once you've scheduled enough pods to exhaust that pool, the VPC CNI has to register and attach an entirely new ENI — and that's not instant; it can take 10-15 seconds, sometimes longer, especially if the subnet itself is running low on free addresses and has to wait for IP leases to expire.

Two knobs help the CNI stay ahead of that churn:

- **WARM_ENI_TARGET** — keep N extra ENIs pre-attached and ready before you need them.
- **WARM_IP_TARGET** — keep N extra IP addresses ready, regardless of how many ENIs that takes (handles mixed instance sizes more gracefully).

Prefix delegation — the fix AWS recommends for everyone

Instead of handing out one IP address per ENI slot, prefix delegation assigns a whole /28 block (16 addresses) as a single route. One ENI slot now unlocks 16 pod IPs instead of one — so a small instance that used to max out at ~4 usable pod IPs can suddenly support ~64. Enable it with the `ENABLE_PREFIX_DELEGATION` environment variable on the VPC CNI.



IPv6: making the whole problem disappear

If you can start a brand-new cluster on IPv6 (dual-stack VPC), running out of IPs stops being a thing you worry about — each node gets an /80 block, which is 100 trillion addresses (more than the entire IPv4 internet, which only has about 4 billion addresses, total). Pods only get an IPv6 address in EKS (no dual-stack at the pod level); a local 169.254.x.x address on the node handles NAT translation for the rare case where a pod still needs to call a legacy IPv4 endpoint. One thing to watch: many pods on one node all sharing that single outbound IPv4 NAT path can become a bottleneck if a lot of your surrounding infrastructure is still IPv4-only.

Network policies

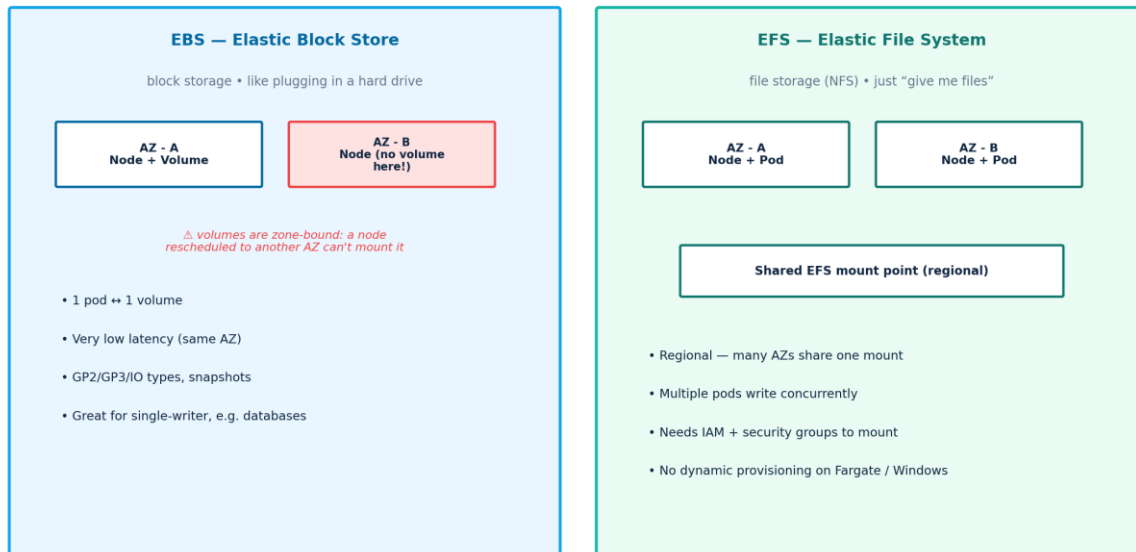
Network policies are the standard Kubernetes way to control which pods can talk to which other pods (ingress and egress). The AWS VPC CNI implements them using eBPF — a small program running in the node's Linux kernel, not a sidecar — so it's a genuinely application-native firewall. The big benefit: application teams can manage their own traffic rules as part of the same manifests they deploy, instead of filing a ticket with a separate network/firewall team every time an endpoint changes.

3. Storage

EBS, EFS, and where your data actually lives

Kubernetes constructs like `emptyDir` are fine for scratch space, but they disappear the moment the pod does. For anything durable — like a `StatefulSet`'s data — you need storage that survives outside the pod's lifecycle. In AWS that almost always means EBS or EFS.

Two main ways to get durable storage into a pod



Both are wired into the cluster through a CSI driver (Container Storage Interface). Each driver ships two pieces: a controller Deployment that manages the full lifecycle of volumes (creating/deleting/tracking them against the AWS API), and a DaemonSet that runs on every node to actually perform the mount, similar to how the VPC CNI drops files in place for the kubelet.

- **EBS** is zone-bound — if your autoscaler creates a replacement node in the wrong AZ, the pod will sit unschedulable while it cycles through availability zones trying to find the right one. It's fast, supports snapshots, and is the natural fit for single-writer workloads like databases.
- **EFS** is a regional NFS share — multiple pods across multiple AZs can mount the same volume simultaneously. Great for shared config or shared uploads, but be aware of classic NFS pitfalls (file locking can hang concurrent writers), and dynamic provisioning isn't available on Fargate or Windows nodes.

Beyond EBS/EFS, you also have FSx for Lustre (high-throughput HPC-style workloads), S3 (with a newer S3 Mount Point CSI driver — handy, but not POSIX-compliant, so things like renames or ``mkdir`` behave differently than you'd expect), local NVMe drives (blazing fast, but ephemeral), and in-cluster options like Longhorn, Rook/Ceph, or OpenEBS for teams that want storage to be fully portable across clouds or on-prem.

A quick word on Secrets

A Kubernetes Secret looks like a config map, except the value is base64-encoded before it's sent to the API server. It's important to internalize: base64 is not encryption. It's trivially reversible — it just obscures the value from a casual glance, it doesn't protect it. For anything genuinely sensitive (API keys, root passwords), the recommended pattern is to store secrets outside the cluster — AWS Secrets Manager, or HashiCorp Vault if you're multi-

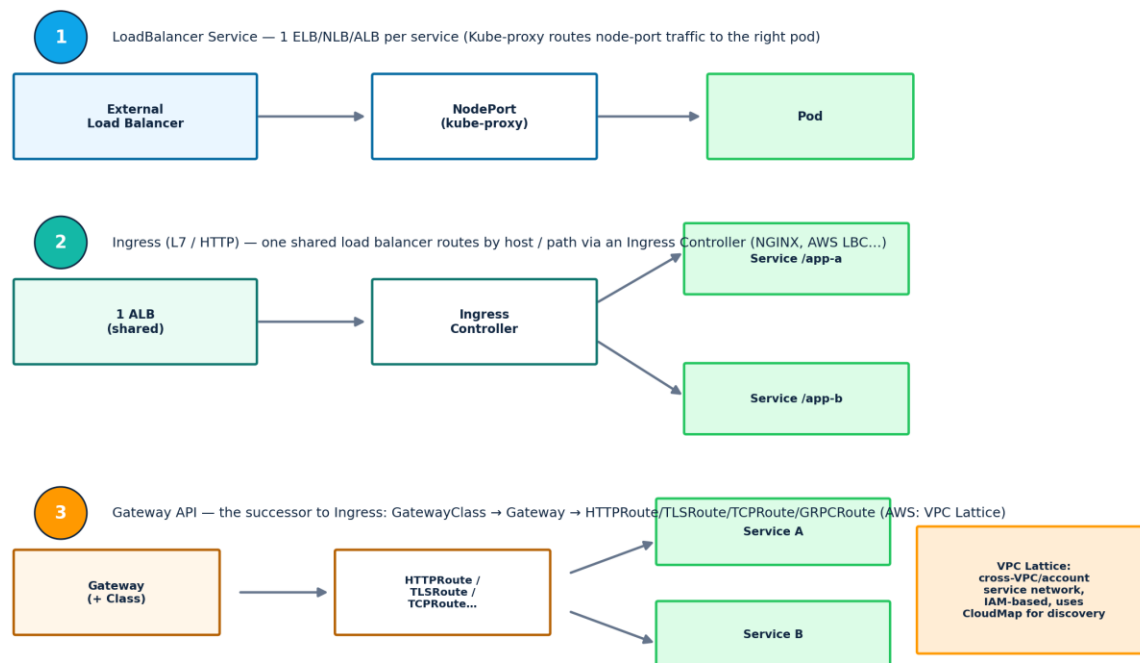
cloud/on-prem — and mount them into pods using the Secrets Store CSI Driver. That driver can also sync short-lived Kubernetes Secrets for you, and handles rotation, so your workload always reads the latest value without ever needing AWS SDK credentials baked into the application itself.

4. Getting Traffic In & Out

Services, load balancers, and Ingress

Kubernetes originally had cloud-provider logic baked right into the control plane; today that's a separate cloud controller (EKS runs it for you) plus, for load balancing specifically, the AWS Load Balancer Controller. Understanding this stack starts with understanding what a Service actually does under the hood: it exposes a NodePort on every node in the cluster, whether or not that node is running a matching pod. If a request lands on a node without the pod, kube-proxy quietly reroutes it to a node that does have it — which is why AWS recommends setting externalTrafficPolicy to only route to nodes actually running the pod, saving that extra hop.

Getting traffic in: three layers, from simplest to most flexible



- **LoadBalancer Service** — the simplest option: one dedicated ELB/NLB/ALB per Kubernetes Service. Fine at small scale, expensive and noisy at large scale (hundreds of load balancers to manage).
- **Ingress** — layer-7 (HTTP) routing by hostname or path, behind a single shared load balancer. Historically this meant running NGINX as an Ingress Controller inside the cluster; today the AWS Load Balancer Controller can create and manage an ALB directly from Ingress resources — and multiple Ingress objects can even share the same ALB to save cost.

⚠ *Ingress is officially not graduating to a stable Kubernetes API — it's being succeeded by the Gateway API. The AWS Load Balancer Controller does not (as of this course) support Gateway resources, so if you need Gateway-native routing today, look at AWS's newer VPC Lattice controller instead.*

One more piece worth knowing: External DNS is a popular open-source controller that watches your Services/Ingresses and automatically creates matching Route 53 records pointing at the right load balancer — no manual Terraform or console clicking required. And for multi-region setups, AWS's Global Load Balancer sits outside any single region and can split traffic across regional load balancers by geography or weighted percentage (though it isn't yet managed by the AWS Load Balancer Controller itself).

5. The Gateway API & VPC Lattice

The next generation of Ingress

Think of the Gateway API as “Ingress v2”: same idea (route layer-7 traffic into the cluster), but with far more flexibility and protocol support. Instead of one Ingress resource doing everything, routing is split into composable pieces:

- **GatewayClass** — groups gateways by which controller manages them (similar to IngressClass).
- **Gateway** — the actual listener; points to a backend Service.
- **HTTPRoute / TLSRoute / TCPRoute / UDPRoute / GRPCRoute** — separate, purpose-built ways to describe how traffic should be matched and routed, instead of Ingress's one-size-fits-all HTTP host/path matching. TLSRoute, for example, routes purely on the SNI hostname from the TLS handshake, without terminating TLS itself.

VPC Lattice

AWS's implementation for Gateway API traffic is VPC Lattice — a network-meshing service that sits transparently below (or above, depending how you look at it) your VPCs. Its core abstraction is a service network: a group of service endpoints, registered via AWS Cloud Map (conceptually similar to how kube-dns works inside a cluster), that can reach each other as long as IAM allows it — regardless of which VPC or even which AWS account they live in.

Lattice can connect not just Kubernetes Services but also Lambda functions and EC2 instances into the same mesh, which is genuinely powerful for large, multi-account organizations. But it comes with real trade-offs:

- Everything is IAM-gated, so you'll lean on IAM policy far more heavily than a typical Kubernetes network policy setup.
- Provisioning a service network can take 5–10 minutes — a jarring wait if you're used to Kubernetes resources appearing almost instantly.

📌 This is squarely an advanced, large-enterprise tool. If you're running one or two clusters, there are much simpler ways to route traffic between services — save Lattice for when you're dealing with dozens or hundreds of VPCs and accounts.

6. Compute Options

Fargate, Node Groups, and Karpenter

Every EKS cluster needs somewhere for pods to actually run. There are three main paths, and they solve different problems.

Three ways to get compute (data plane) into an EKS cluster

Fargate	Node Groups	Karpenter
• “Serverless” — no EC2 instance visible in your account • 1 pod = its own isolated node • No DaemonSets → convert to sidecars • No dynamic EBS / EFS volumes • Best for: isolated / security-sensitive workloads, or core cluster services (metrics-server, autoscaler)	• Backed by an EC2 Auto Scaling Group • Managed = AWS handles AMI/version rollout; Unmanaged = you do everything • Custom AMIs supported (via launch templates) • Upgrades roll instances one/N at a time • Best for: predictable, steady-state workloads — the classic default	• Kubernetes-native autoscaler — no ASGs, no pre-defined node groups • Reads unscheduled pods → picks the cheapest EC2 type that fits, on demand • “Consolidation” bin-packs / replaces nodes to cut cost continuously • Needs mature workloads: PodDisruptionBudgets, topology spread, resource requests • Best for: dynamic, cost-sensitive, mixed workloads

Fargate

When Fargate is enabled, a mutating webhook swaps in a special Fargate scheduler for matching pods — the standard Kubernetes scheduler has no concept of Fargate at all. That Fargate scheduler calls out to AWS, provisions a right-sized node just for that one pod (adding overhead for the kubelet and any sidecars), and binds the pod to it. Crucially, that node never shows up as an EC2 instance in your account — that's the “serverless” part. The trade-off is real isolation: DaemonSets aren't supported at all (you have to convert them into sidecars, which multiplies your resource usage at scale), and neither are EBS volumes or dynamic EFS provisioning. Fargate shines for workloads that genuinely need dedicated, isolated compute — or for core cluster services like the metrics-server or autoscaler, which benefit from being insulated from the churn of regular node upgrades.

Node Groups

Node groups are just a way to group EC2 instances (backed by an Auto Scaling Group) that all belong to the cluster. Unmanaged node groups mean you handle everything yourself — the ASG, joining the kubelet to the cluster, IAM credentials, all of it. Managed node groups let AWS handle version upgrades for you: trigger an upgrade, and EKS rolls new instances in via the launch template while draining old ones out. Expect real churn during this process — workloads often get rescheduled three, four, five times as they bounce between old and new instances before finally landing on a fully-upgraded node.

Karpenter

Karpenter, originally built by AWS and donated to the Kubernetes autoscaling SIG, takes a fundamentally different approach: no pre-declared node groups at all. When a pod is unschedulable, Karpenter looks at every EC2 instance type available in your region (not just what's in a pre-defined group), finds the cheapest one that satisfies the pod's requirements, and provisions it on demand. It also does continuous “consolidation” — constantly re-evaluating whether it can bin-pack workloads onto fewer or cheaper nodes and swap accordingly.

⚠ *Karpenter's biggest catch: your workloads need to be mature. Without PodDisruptionBudgets, topology spread constraints, and proper resource requests defined, Karpenter will happily schedule everything onto a single giant (cheapest) instance and reshuffle nodes aggressively — which, without those guardrails, is a great way to cause an outage.*

7. Cluster Access & Authentication

How AWS identities become Kubernetes permissions

When you run `kubectl get nodes`, the request goes through an AWS-managed Application Load Balancer in front of the control plane. Your kubeconfig invokes the `aws-iam-authenticator` plugin, which fetches AWS credentials for you and presents them to that ALB. But AWS IAM identities and Kubernetes RBAC roles are two completely separate systems — something has to map one to the other.

- **aws-auth ConfigMap (legacy)** — the original approach: a ConfigMap living in kube-system that manually maps IAM ARNs to RBAC roles. It's plain text, so a single formatting mistake could lock you out entirely, and whoever created the cluster was always silently granted permanent cluster-admin — with no clean way to revoke it if that IAM identity was ever deleted.
- **EKS Access Entries API (current)** — the modern replacement. Access mappings are now managed declaratively through the EKS API itself, not a fragile in-cluster file. Cluster creation and cluster administration can be cleanly split across different teams, and it's much safer to automate.

📌 aws-auth still works and still exists — but if you're setting up a new cluster today, use the EKS Access Entries API instead.

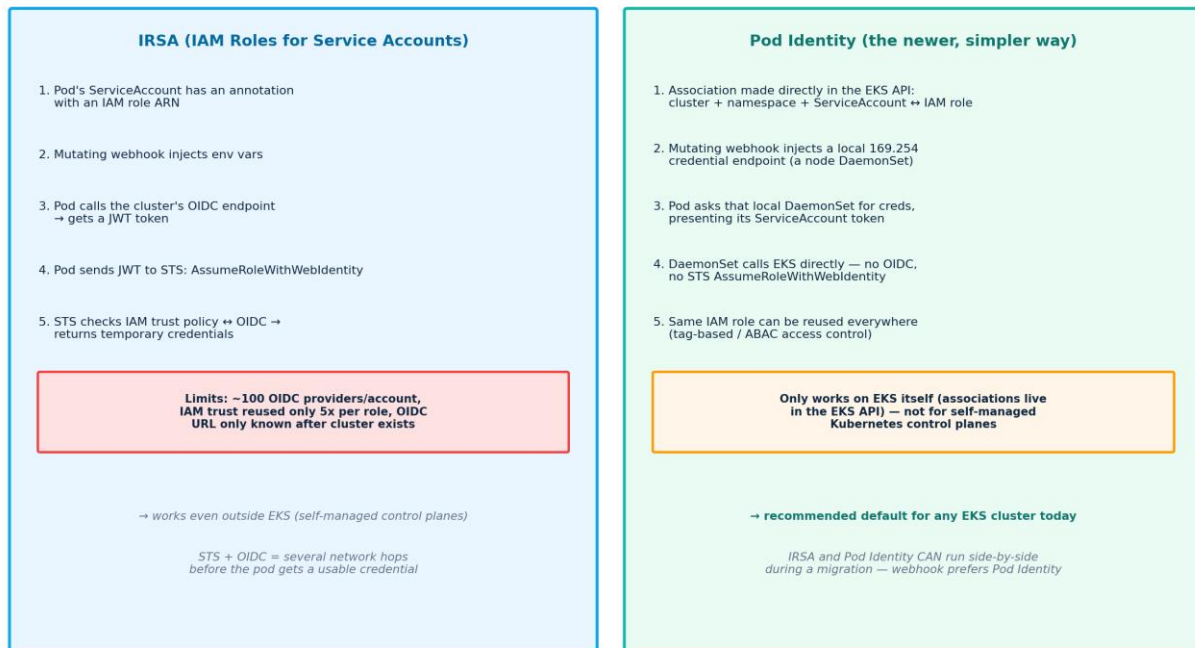
8. IAM for Pods

Kube2IAM, IRSA, and Pod Identity

Cluster-level access (Section 7) answers “can a human or CI system talk to the API server?” This section answers a different question: “how does a pod get permission to call an AWS API, like reading from an S3 bucket?” There have been three generations of answers.

- **Kube2IAM** — an older, community pattern that intercepts calls to the EC2 metadata endpoint and redirects them based on pod annotations. It works, but it's awkward: Kubernetes-native RBAC uses ServiceAccounts, while Kube2IAM permissions live as annotations directly on the workload — two different mental models mixed together.

Getting AWS credentials into a pod: IRSA vs. Pod Identity



- **IRSA (IAM Roles for Service Accounts)** — the first AWS-native solution. It relies on the cluster's OIDC endpoint: a pod fetches a JWT, exchanges it with STS for temporary credentials, and STS validates the request against an IAM trust policy tied to that OIDC provider. It works even outside EKS, but comes with real limits — a hard cap of roughly 100 OIDC providers per AWS account, IAM trust relationships that can only be reused about 5 times per role, and the fact that you can't pre-create trust relationships until the cluster (and its OIDC URL) already exists.
- **Pod Identity** — the current, recommended approach. The mapping between a ServiceAccount and an IAM role is stored directly in the EKS API (no OIDC, no STS AssumeRoleWithWebIdentity needed). A DaemonSet on each node hands out credentials locally, and because AWS trusts the EKS service principal itself, the same IAM role can be reused everywhere — access is scoped using tag-based (ABAC) policies instead of juggling dozens of near-identical roles.

📌 If you're running plain EKS today, use Pod Identity. IRSA and Pod Identity can actually run side-by-side during a migration — the injection webhook simply prefers Pod Identity when both are enabled for a workload.

9. Security Groups for Pods

A powerful option — used sparingly, on purpose

By default, every ENI on a node — and therefore every pod scheduled to that node — shares the same security group. If any workload on a node needs to reach an RDS instance, every pod on that node technically has the network path to try (IAM is still what actually stops unauthorized access at the application layer).

AWS does offer a way to assign distinct security groups per pod, using a private-API VPC controller that attaches special “trunk” ENIs capable of carrying multiple security groups down to individual pods. It works — but the course is refreshingly blunt about the trade-offs:

- Only some EC2 instance types support trunk ports, and there's no API to check — you have to consult a lookup table.
- Pods using trunk-port security groups aren't counted the same way in the VPC CNI's max-pods calculation, so you end up manually adjusting node capacity.
- Combining this with a non-default CNI (Calico, Cilium) multiplies the complexity dramatically.

📌 *The recommended alternative stack: Pod Identity for AWS-level access control, Kubernetes Network Policies for coarse IP/CIDR-based traffic rules, and separate node groups (or even separate clusters) when you need a hard security boundary around especially sensitive data. Security groups for pods is a real feature — but reach for it only if you already have deep in-house security-group tooling and expertise.*

10. Observability & Monitoring

Almost everything eventually lands in CloudWatch

Control plane logging is a simple opt-in checkbox at cluster-creation time — flip it on, and API server / scheduler / controller-manager logs start flowing into a CloudWatch log group automatically. For your nodes and workloads, EKS layers a few optional add-ons on top of each other:

- **CloudWatch Observability add-on** — installs the CloudWatch agent as a DaemonSet, enriching node and container logs with extra metrics.
- **ADOT (AWS Distro for OpenTelemetry)** — builds on top of that agent to also export metrics/logs (and traces) to any OpenTelemetry-compatible backend, not just CloudWatch.
- **AWS X-Ray add-on** — builds on top of ADOT specifically for distributed tracing.

Fargate nodes are the exception — since they can't run DaemonSets, logging instead uses a managed Fluent Bit (“Firelens”) integration, triggered by creating an aws-observability namespace and a logging ConfigMap; it captures workload logs, not rich node-level metadata (because there isn't really a “node” to inspect).

If you'd rather not build every dashboard in CloudWatch, AWS also offers fully managed Prometheus (AMP) and Grafana (AMG) — the same open-source stacks you'd run anywhere else, but with IAM baked in for auth and access control. AMG's default plugin still tends to nudge you back toward CloudWatch as the primary data source, though.

11. Upgrading Your Cluster

The single most important operational habit with EKS

A standard EKS cluster version is only supported for 14 months. Kubernetes upstream ships three releases a year, and EKS stays in lockstep with that cadence — which, in practice, means you should expect to upgrade roughly every 3-4 months if you want to stay comfortably ahead of the deadline.



Miss the window, and you move into Extended Support: another 12 months of life, but at roughly 5x the hourly cost. AWS's Upgrade Insights feature (in the console or CLI) can tell you which deprecated or soon-to-be-removed APIs your cluster is actively using — genuinely useful, since the team running the cluster is often not the same team that owns the workloads calling those APIs.

In-place vs. blue/green

The course's strong, experience-based recommendation: prefer in-place upgrades almost always. Migrating everything to a brand-new cluster sounds clean in theory, but DNS caching, load balancer cutover, and the sheer number of teams who depend on things “just working” the same way in the new cluster make blue/green migrations take far longer — and cost far more (you're running two clusters at once) — than expected. Reserve blue/green for two specific situations: swapping out your CNI provider entirely, or jumping several major versions at once (which in-place would otherwise force you to do one version at a time).

An in-place upgrade always follows the same order:

- 1. The control plane upgrades first, rolling in a new API server / controller-manager / scheduler behind the load balancer (fully managed by AWS — etcd migrates automatically too).
- 2. Managed node groups roll one (or N, or a %) node at a time via the ASG; Karpenter intelligently drains and replaces (or simply consolidates) nodes as needed; Fargate nodes are replaced by forcing a new Deployment rollout.
- 3. Add-ons must be upgraded individually, per add-on, via their own API calls — this doesn't happen automatically.

📌 Two extra tools worth knowing: the EKS cluster-insights API flags Kubernetes resources that use APIs about to be removed (e.g. PodSecurityPolicies were removed between 1.24 and 1.25); kube-no-trouble (“kubent”) does a similar job with more workload-level detail, at the cost of needing broader RBAC/IAM permissions to inspect every namespace.

12. EKS Add-ons

Convenient — but use with your eyes open

EKS add-ons are a distinctly EKS-specific concept (not a general Kubernetes one): a way to install and version-manage baseline cluster services — things like the VPC CNI, CoreDNS, or the EBS CSI driver — through the EKS API/console instead of a separate Helm install.

The course's take, from someone who worked on EKS add-ons directly, is candid: they're still a bit rough. If you're upgrading a cluster, you end up upgrading the control plane, then your node groups, and then separately upgrading every single add-on — with no single, holistic “test this all together” workflow. The recommendation instead is to own your own baseline YAML — whether that's plain manifests, Helm charts you manage yourself, or a GitOps pipeline — rather than delegating your cluster's foundational services to AWS's add-on marketplace or a third-party vendor's release cadence.

📌 *The broader principle behind this whole section (and honestly, the whole course): the more external dependencies stand between you and an upgrade, the harder every one of those 3-4-times-a-year upgrades becomes. Owning more of your own stack is more work up front, but it keeps you in control of your own upgrade timeline.*

That's the whole course — nice work getting through it!

If any one topic above feels like it deserves its own deep-dive (Karpenter internals, Pod Identity setup, Gateway API routing, whatever), just ask — happy to expand any section with a hands-on walkthrough or more diagrams.